nsdi '26

MAE: More Adaptive Video Encoder for Consistent Low Latency in High-Quality Real Time Communication

__Hua Meng__, Yufan Zhuang, Yasna Noushirvani, Xiangjie Huang, Zili Meng

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Real Time Communication - One of the key uses on the Internet today

Video Conference

Cloud Gaming / PC VR

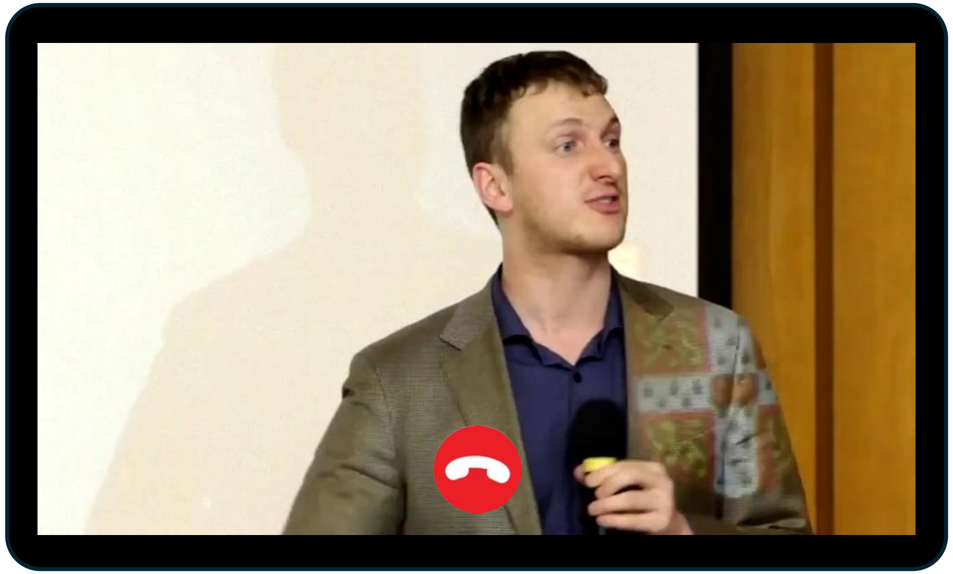
Remote Surgery

The global video conferencing market reached **US$11.65 billion** in 2024, alongside the global cloud gaming market at **US$2.27 billion**.*

* Data from https://www.grandviewresearch.com/industry-analysis

Maintaining consistent low latency is important to user experience

Have you experience this?



99.9th percentile tail latency

↓

Every 1000 frames experience one stall

↓

Every 30 seconds!

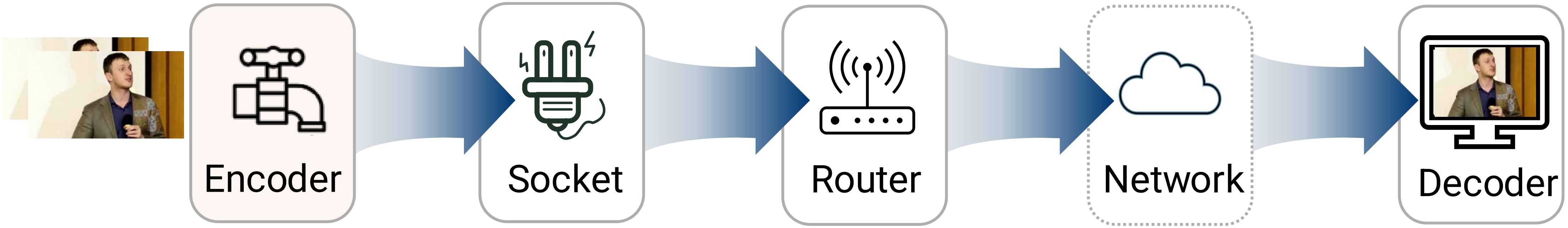
😠 Current **stall** experiences are **still far from satisfactory**.

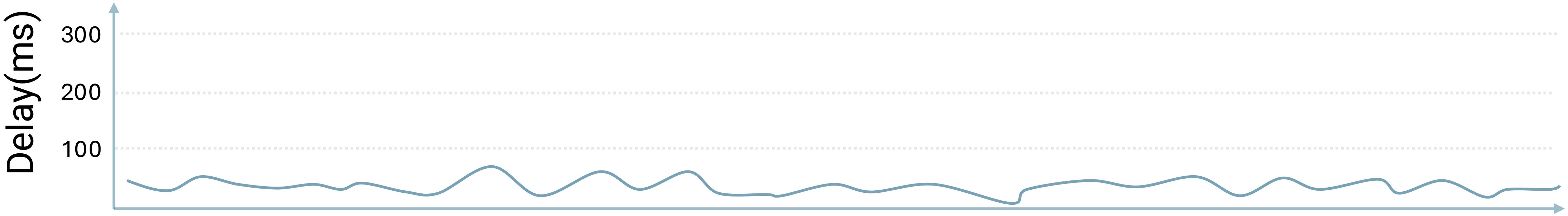
Large-scale live streaming data shows wireless tail latency hits **~200ms** at the **99.9th** percentile.*

* Data from AUGUR, NSDI 2024

Why existing solution failed?

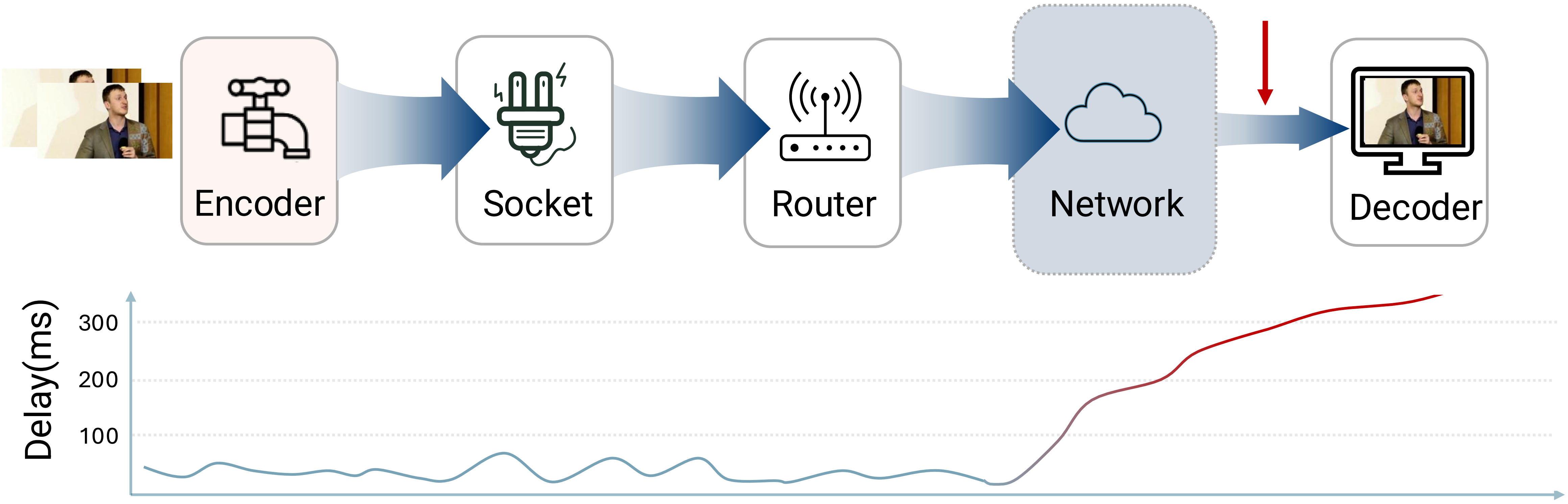
This shows a simplified real-time communication pipeline, which resembles a **water pipeline**.





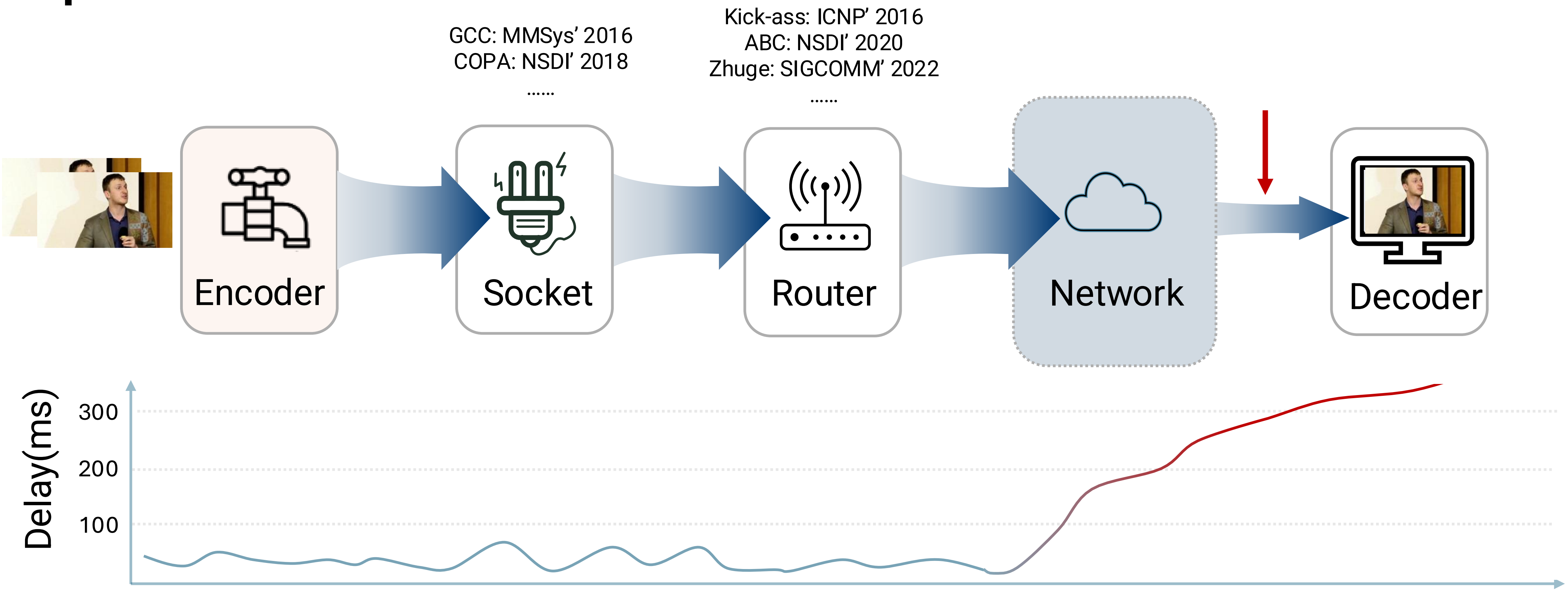
Why existing solution failed?

However, unlike a water pipeline, network bandwidth **fluctuates** dynamically over time.



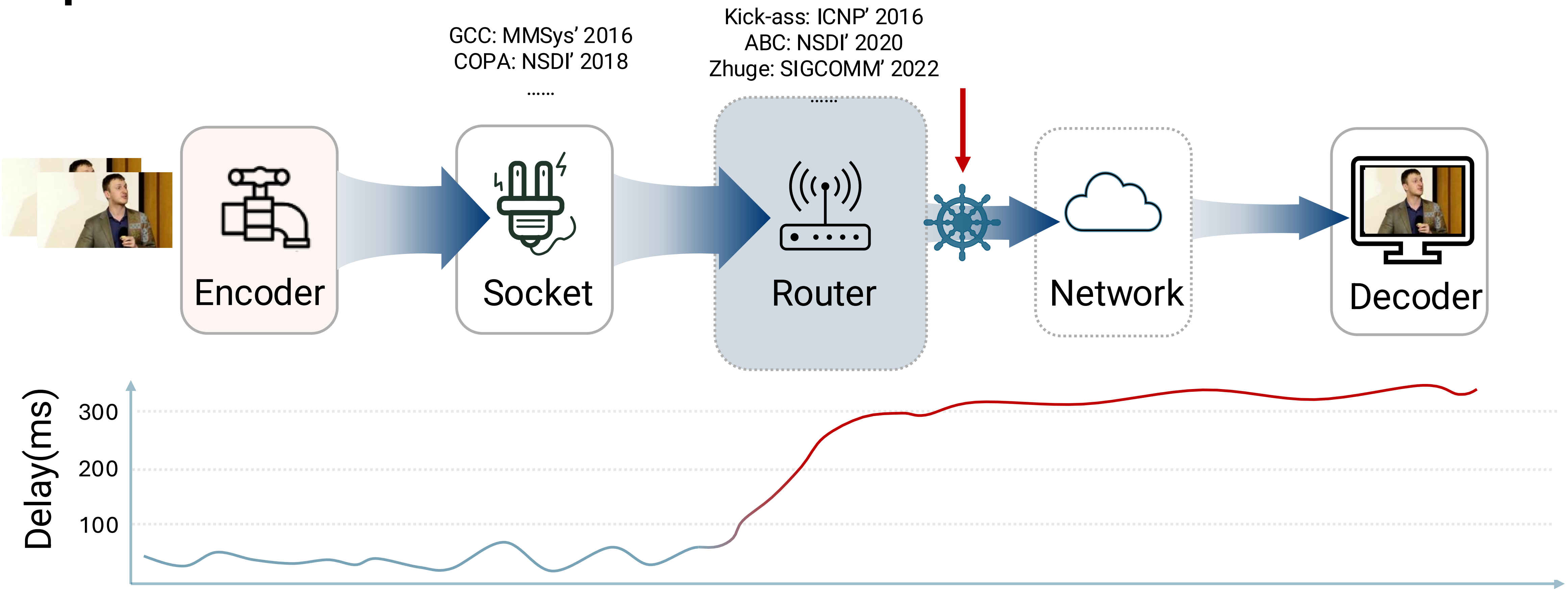
Why existing solution failed?

Previous work simply **shifts** the waiting queue from **downstream to upstream**.



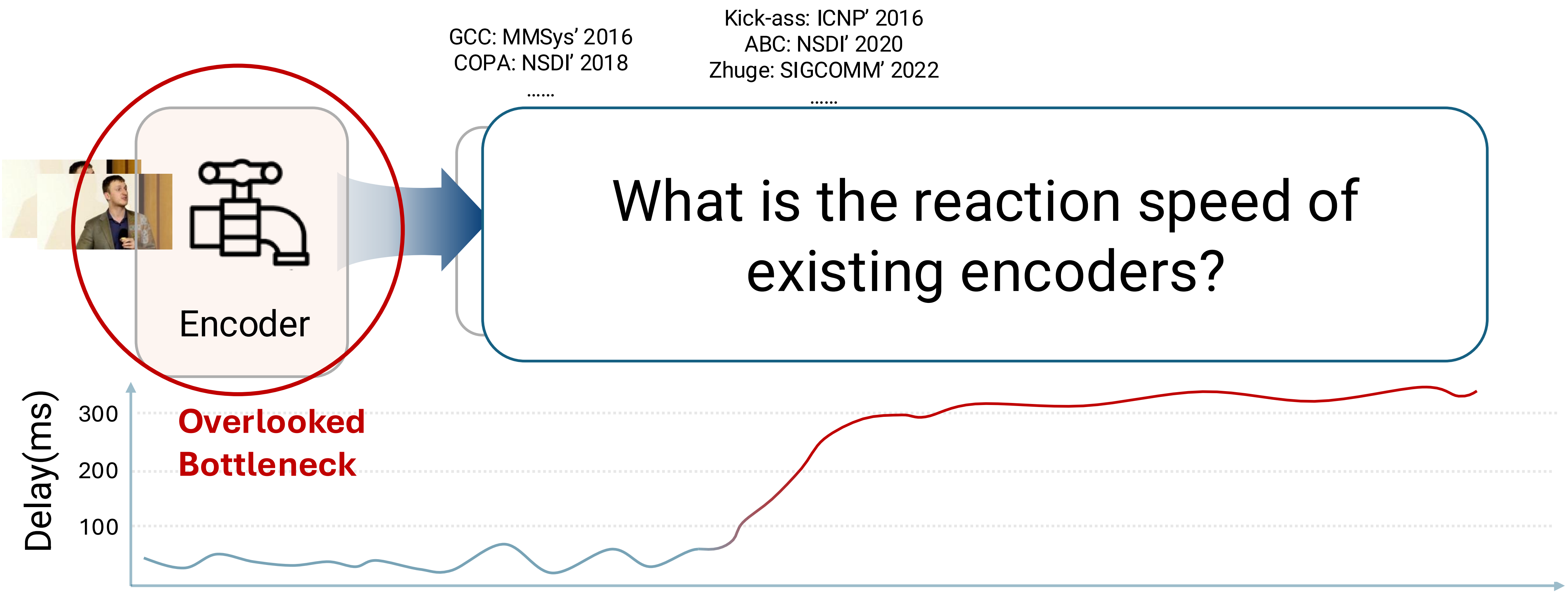
Why existing solution failed?

Previous work simply **shifts** the waiting queue from **downstream to upstream**.



Why existing solution failed?

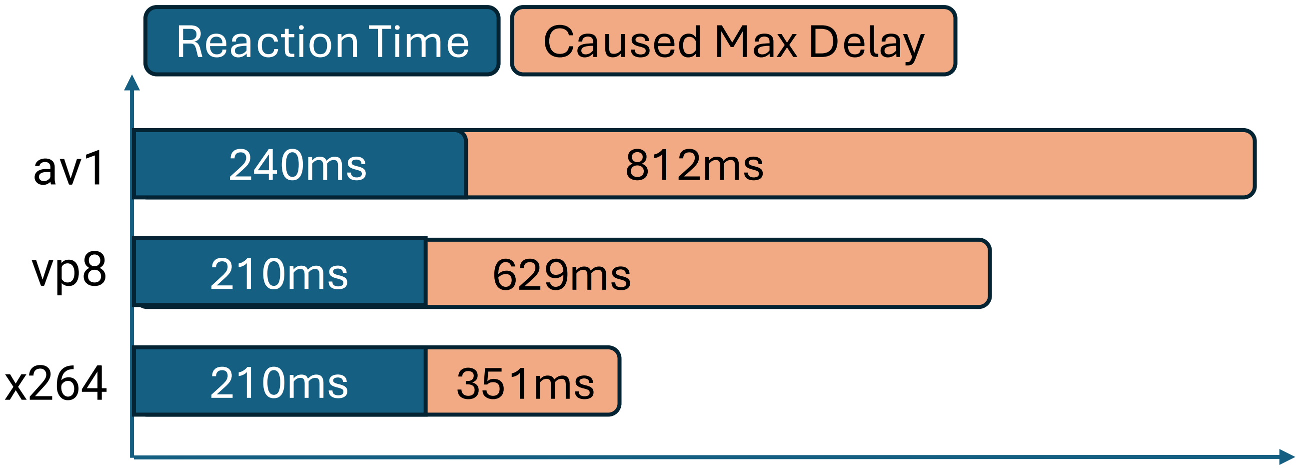
The True Bottleneck: A **Slow-Reacting Source** - Encoder



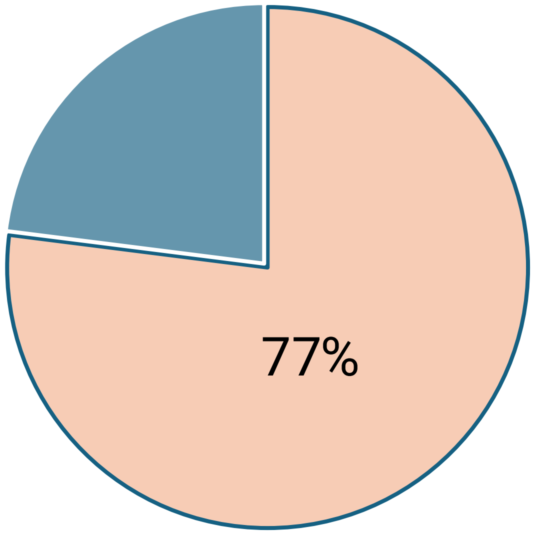
Why existing solution failed?

The True Bottleneck: A **Slow-Reacting Source** - Encoder

What is the reaction speed of existing encoders?



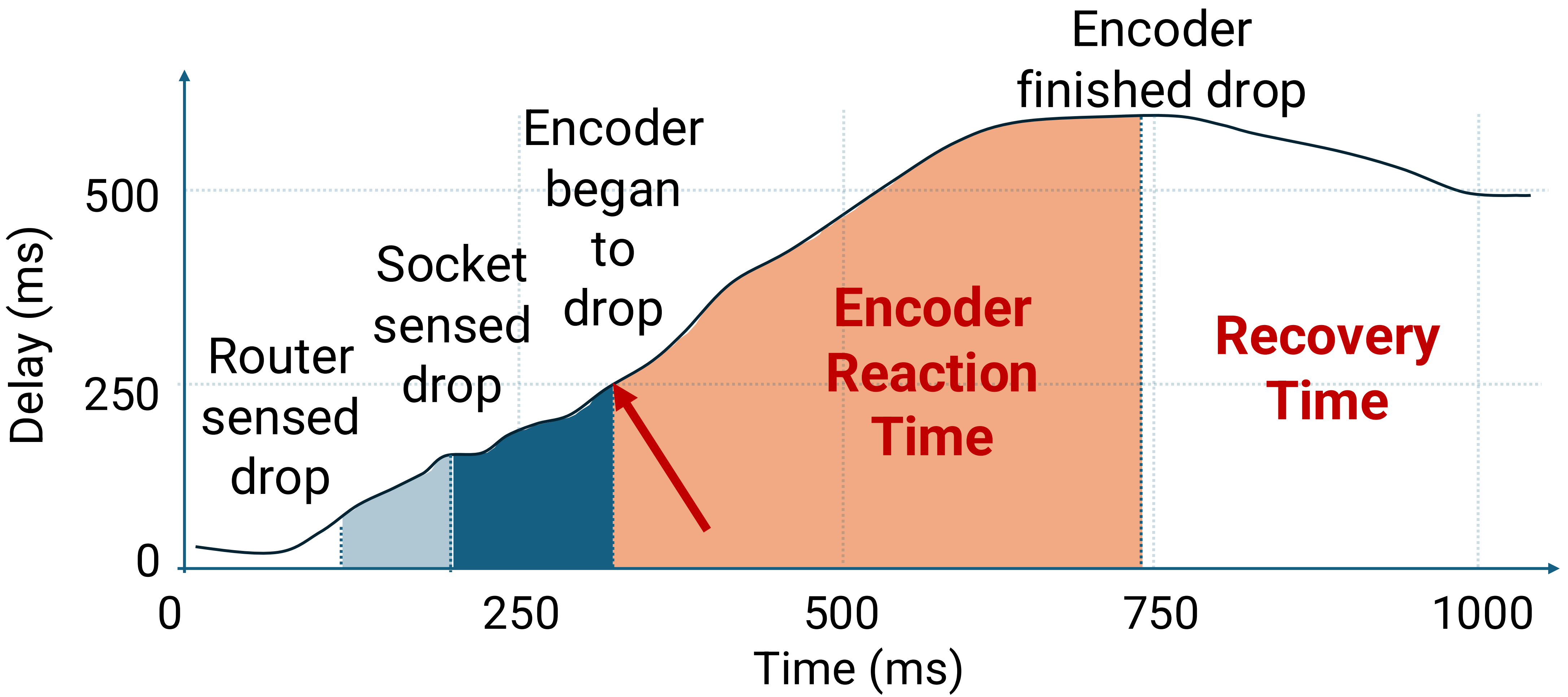
Reaction Speed Under 10x Bandwidth Drop



Our experiment data shows that **77%** reaction time comes primarily from the encoder's reaction time.

Why existing solution failed?

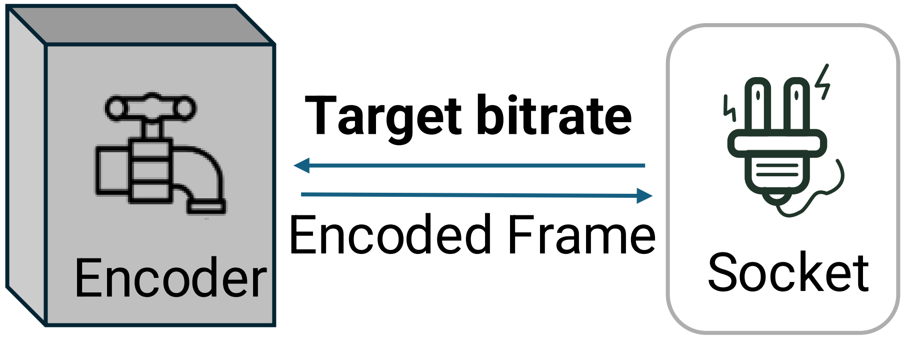
Dive into a detailed bandwidth drop adaptation process.



Detailed reaction time when bandwidth drops from 10Mbps to 1Mbps.

Why current encoders react slowly?

Encoder lacks decision information.

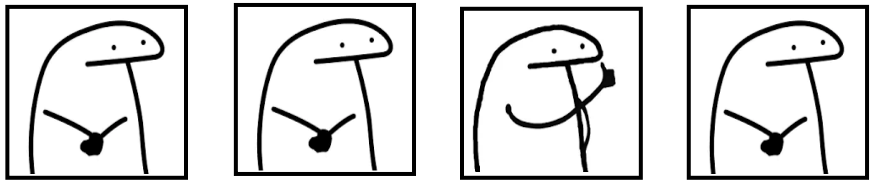


Treated as a black box.

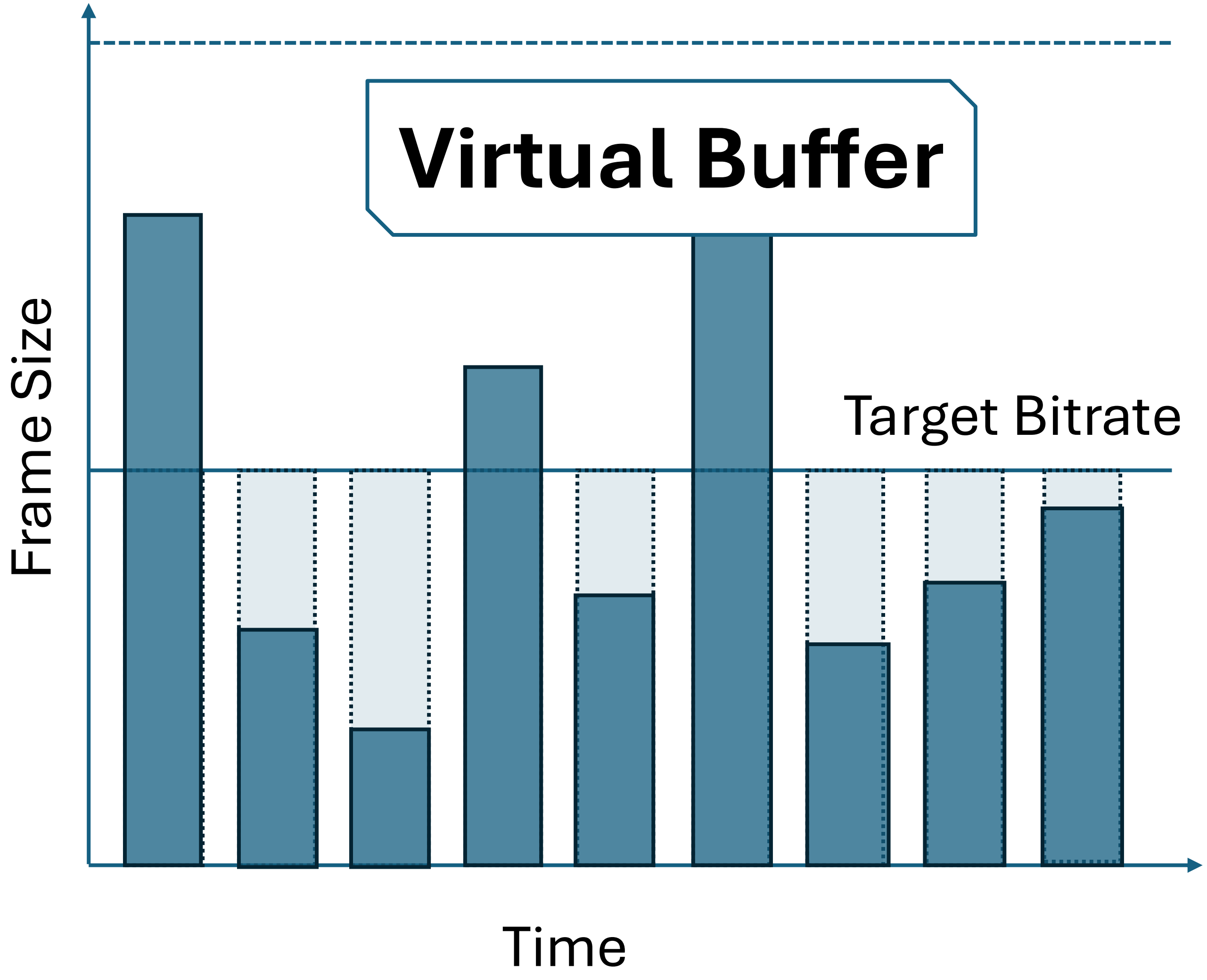
Why current encoders react slowly?

Encoder lacks decision information.

How to control the overshoot?



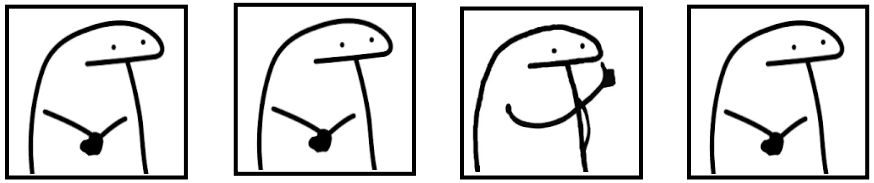
But content is fluctuated.



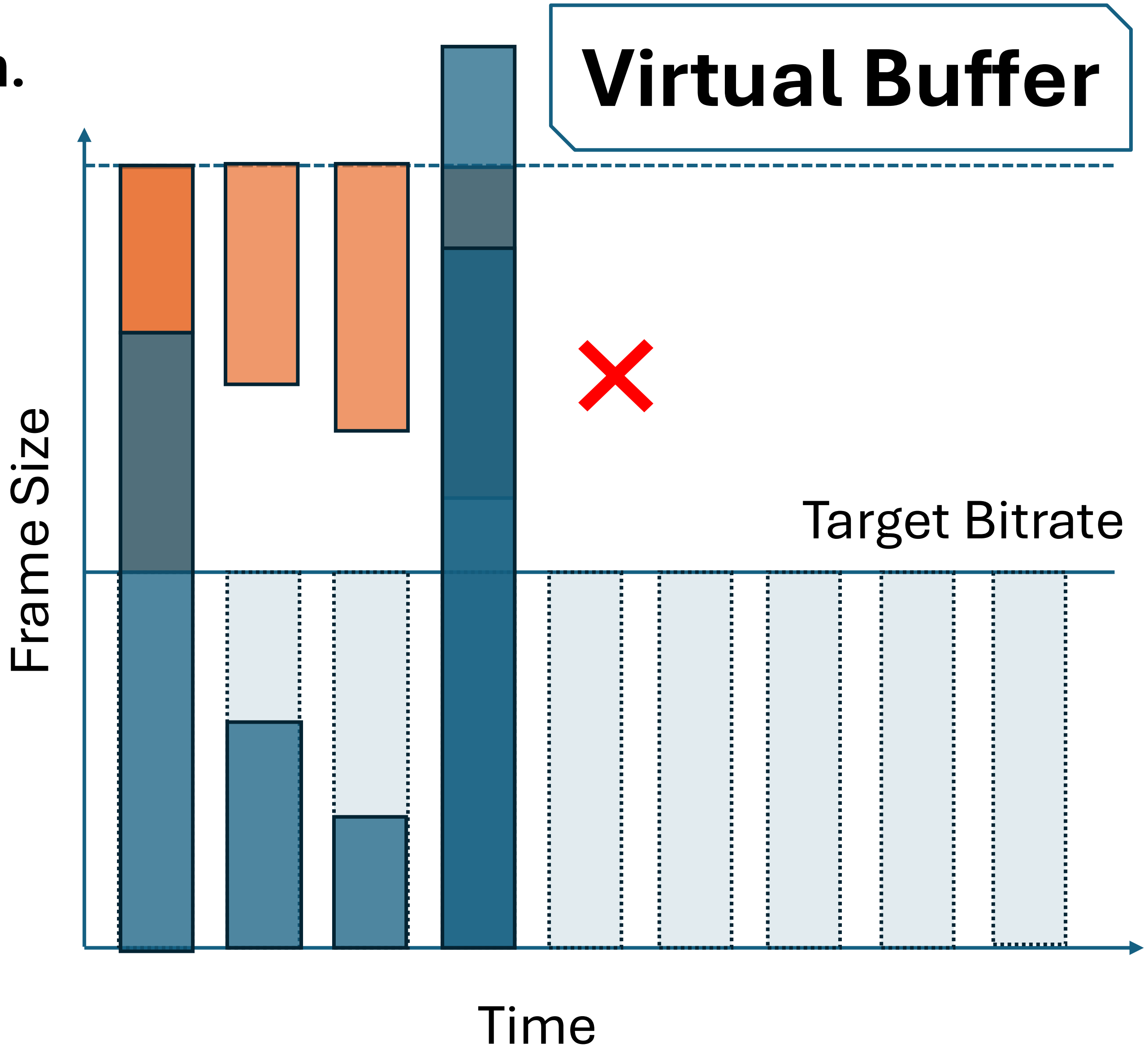
Why current encoders react slowly?

Encoder lacks decision information.

How does the virtual buffer work?



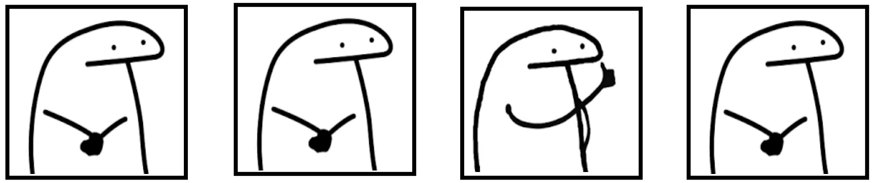
But content is fluctuated.



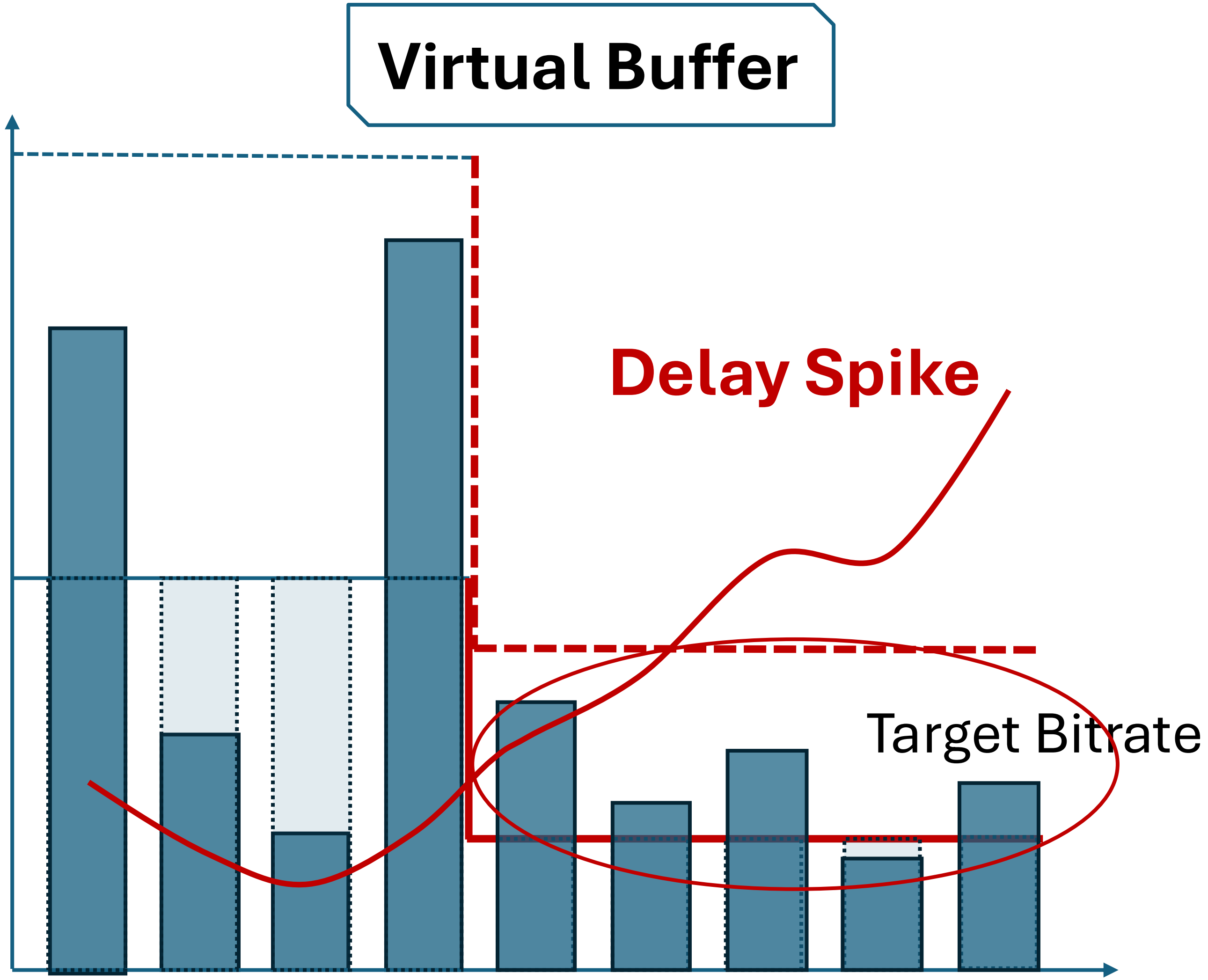
Why current encoders react slowly?

Encoder lacks decision information.

What will happen when bandwidth drops?



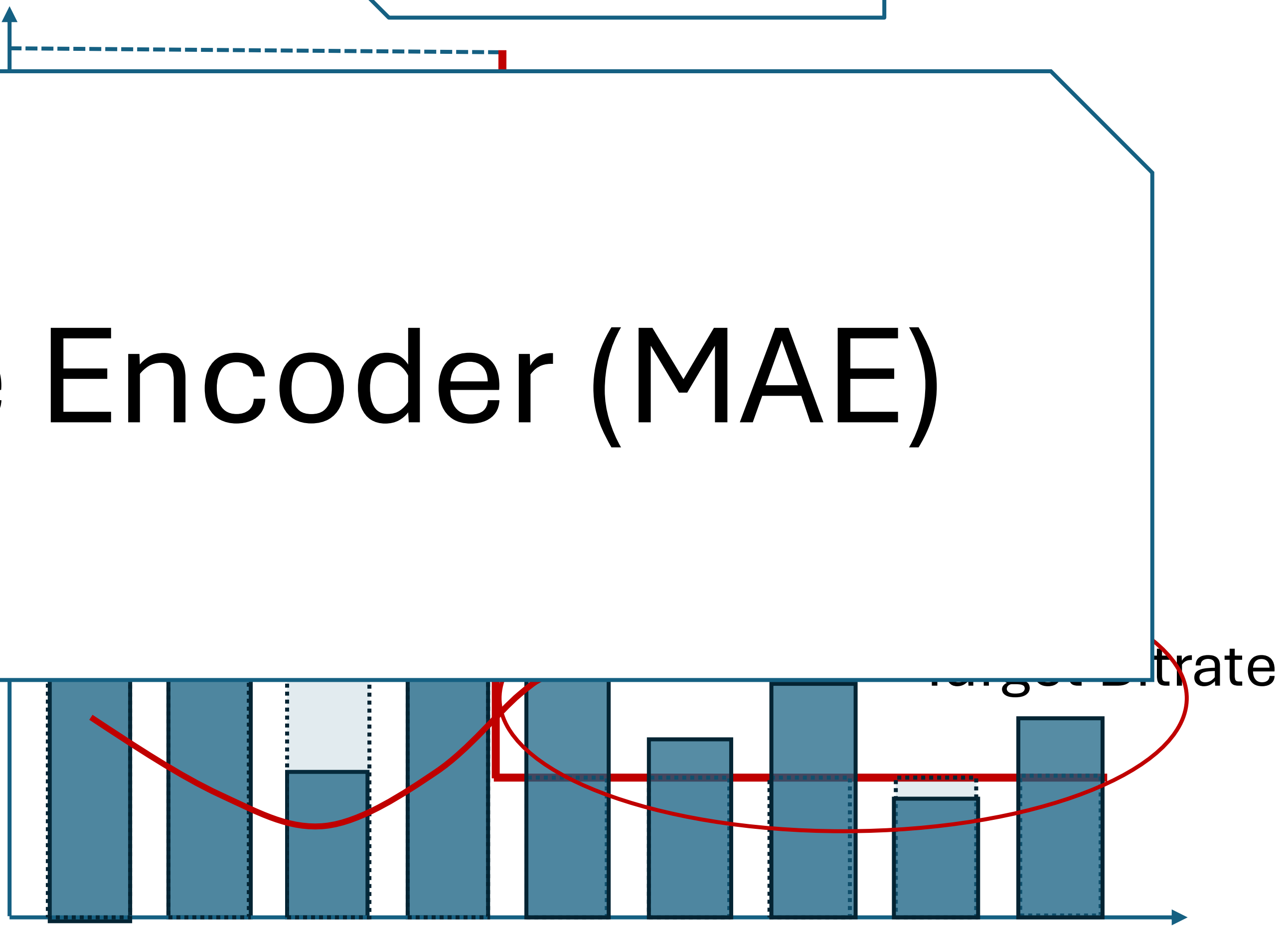
But content is fluctuated.



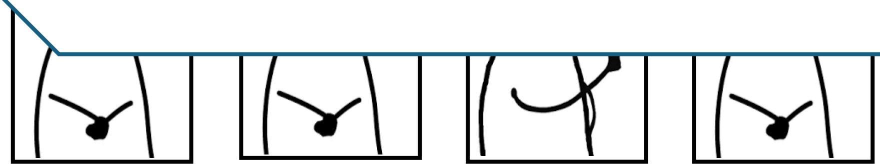
Why current encoders react slowly?

Encoder lacks decision information.

Virtual Buffer

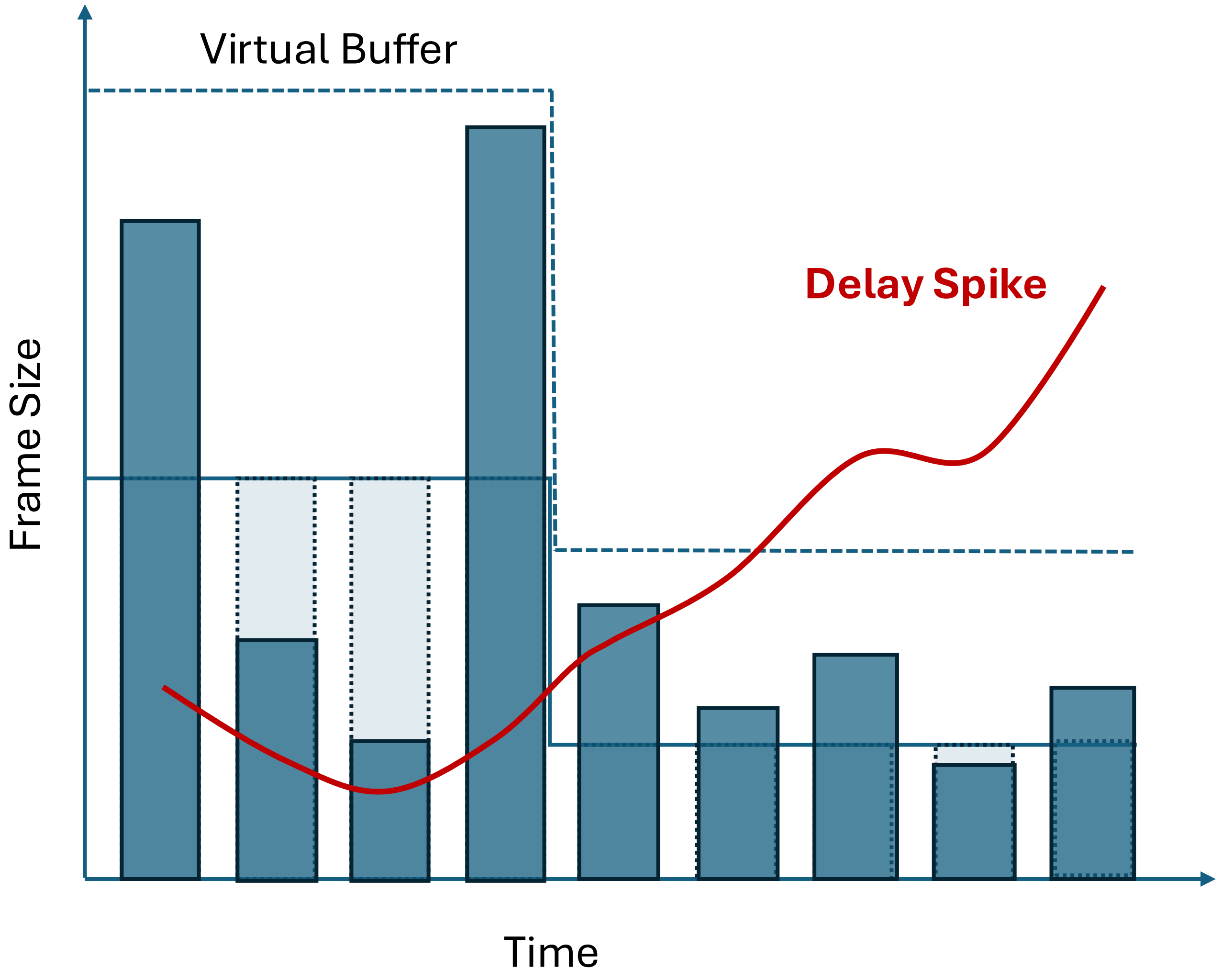


More Adaptive Encoder (MAE)



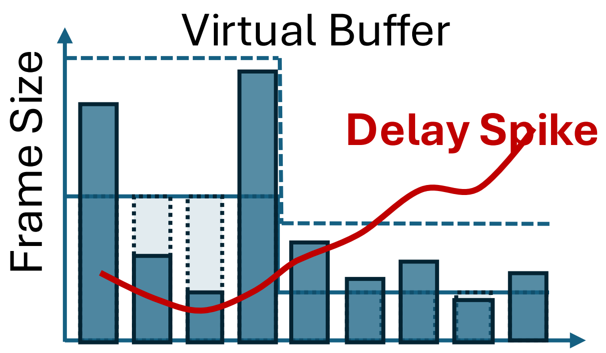
But content is fluctuated.

MAE 1: Dynamic Virtual Buffer Sizing

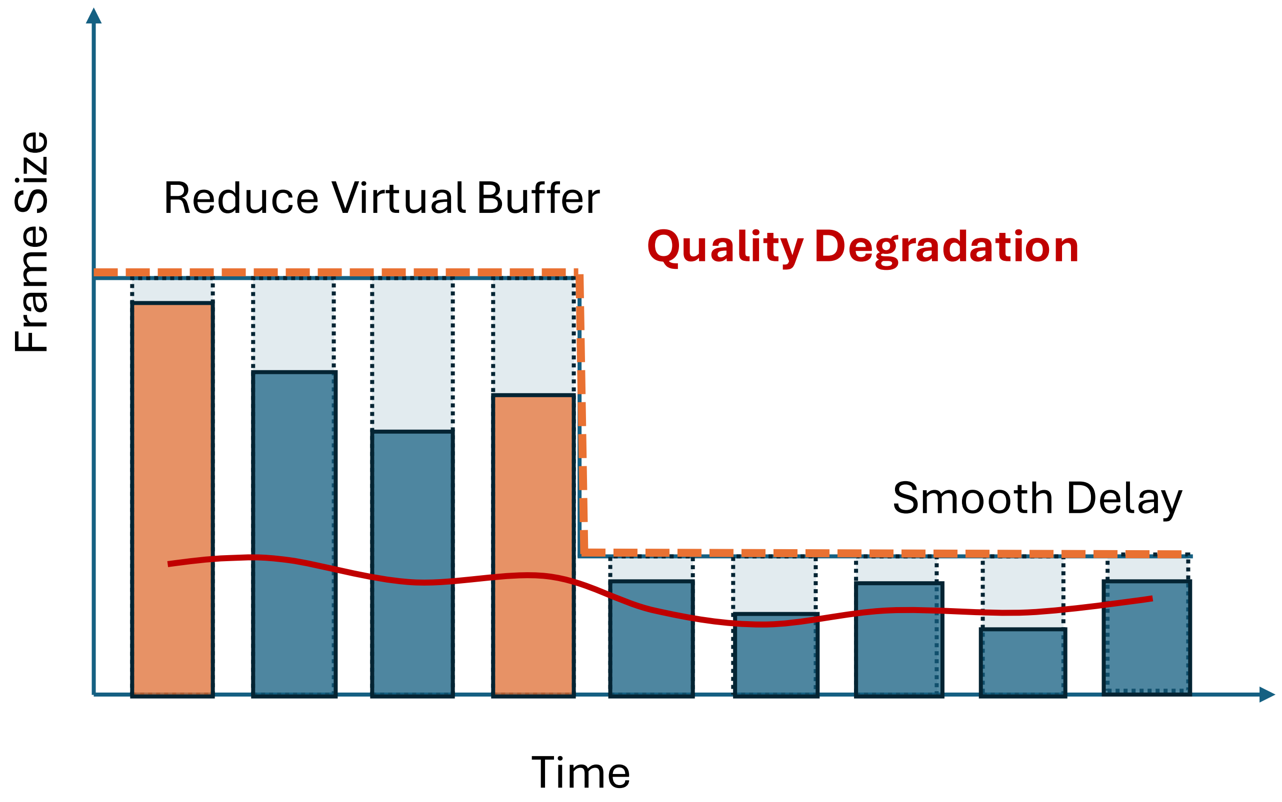


	Always large buffer
Quality	Good
Latency	Bad

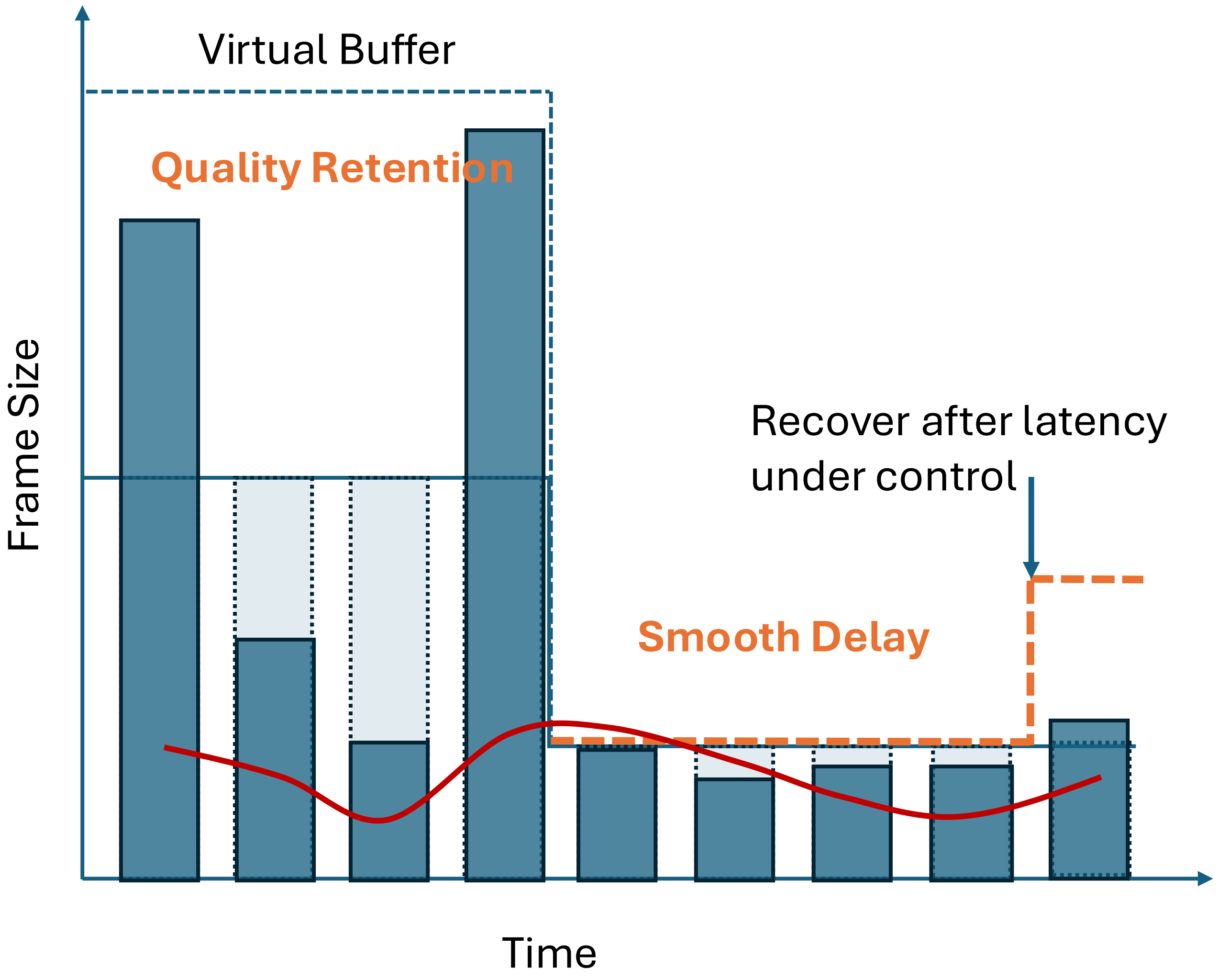
MAE 1: Dynamic Virtual Buffer Sizing

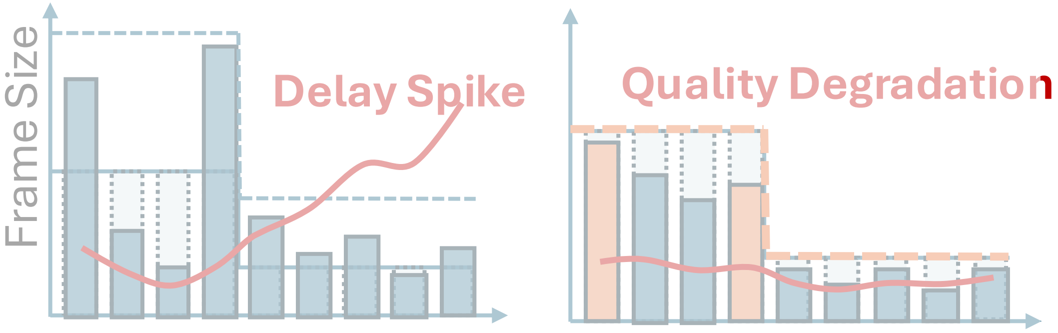


	Always large buffer	Always small buffer
Quality	**Good**	**Bad**
Latency	**Bad**	**Good**



MAE 1: Dynamic Virtual Buffer Sizing

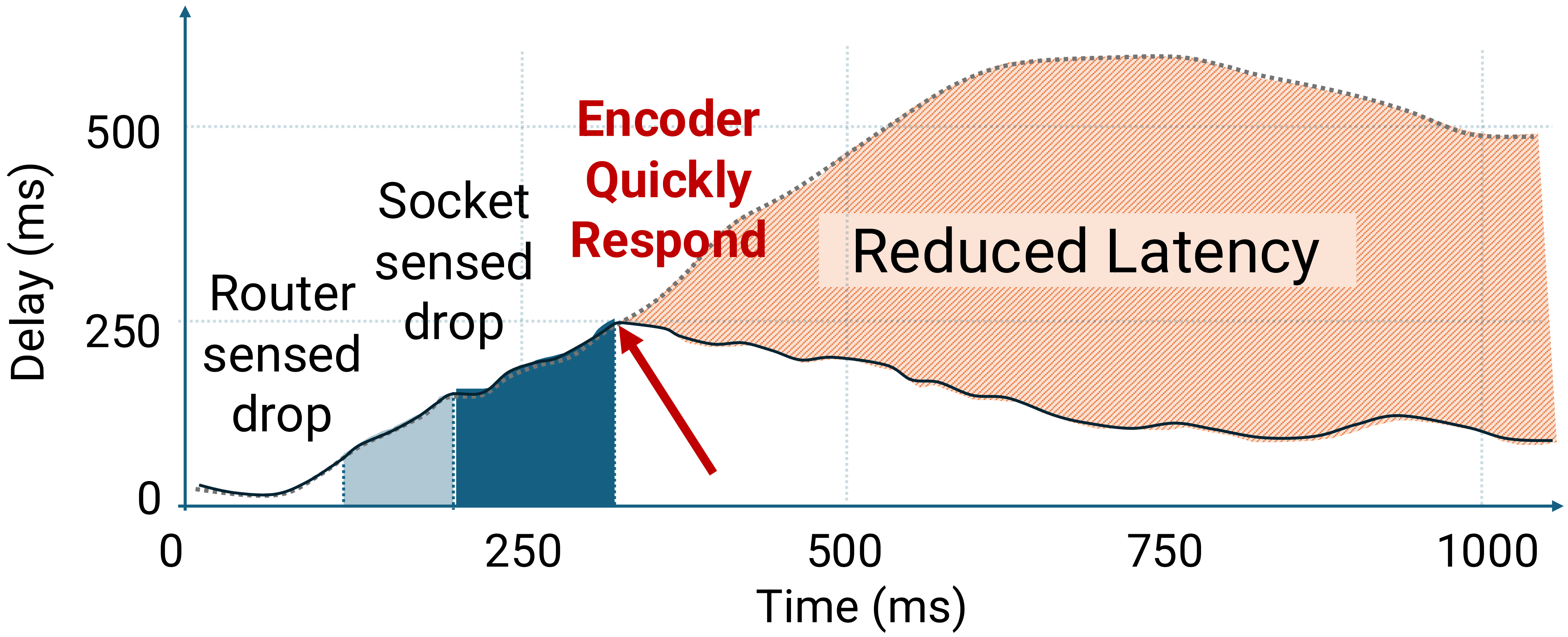




	Always large buffer	Always small buffer
Quality	Good	Bad
Latency	Bad	Good

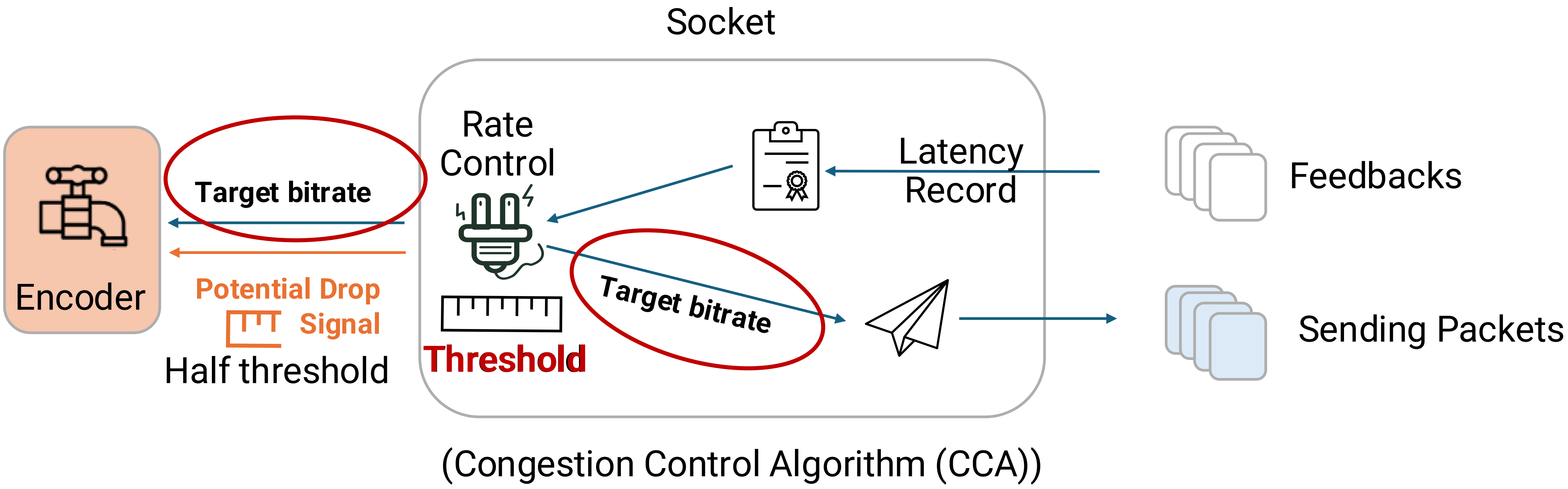
MAE **adapts** it for different network condition.

MAE 1: Dynamic Virtual Buffer Sizing



Detailed reaction time when bandwidth drops from 10Mbps to 1Mbps.

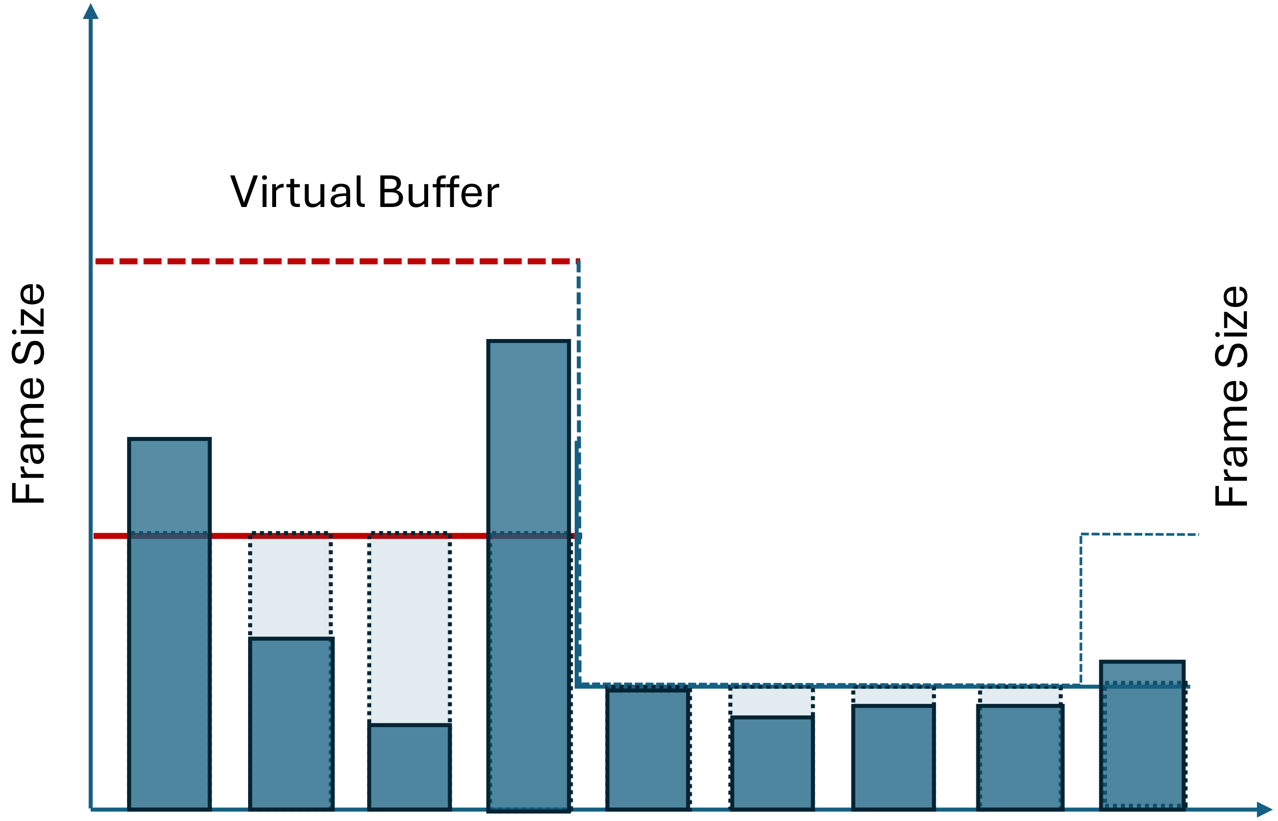
MAE 2: Pre-reacting based on CCA internal state probing



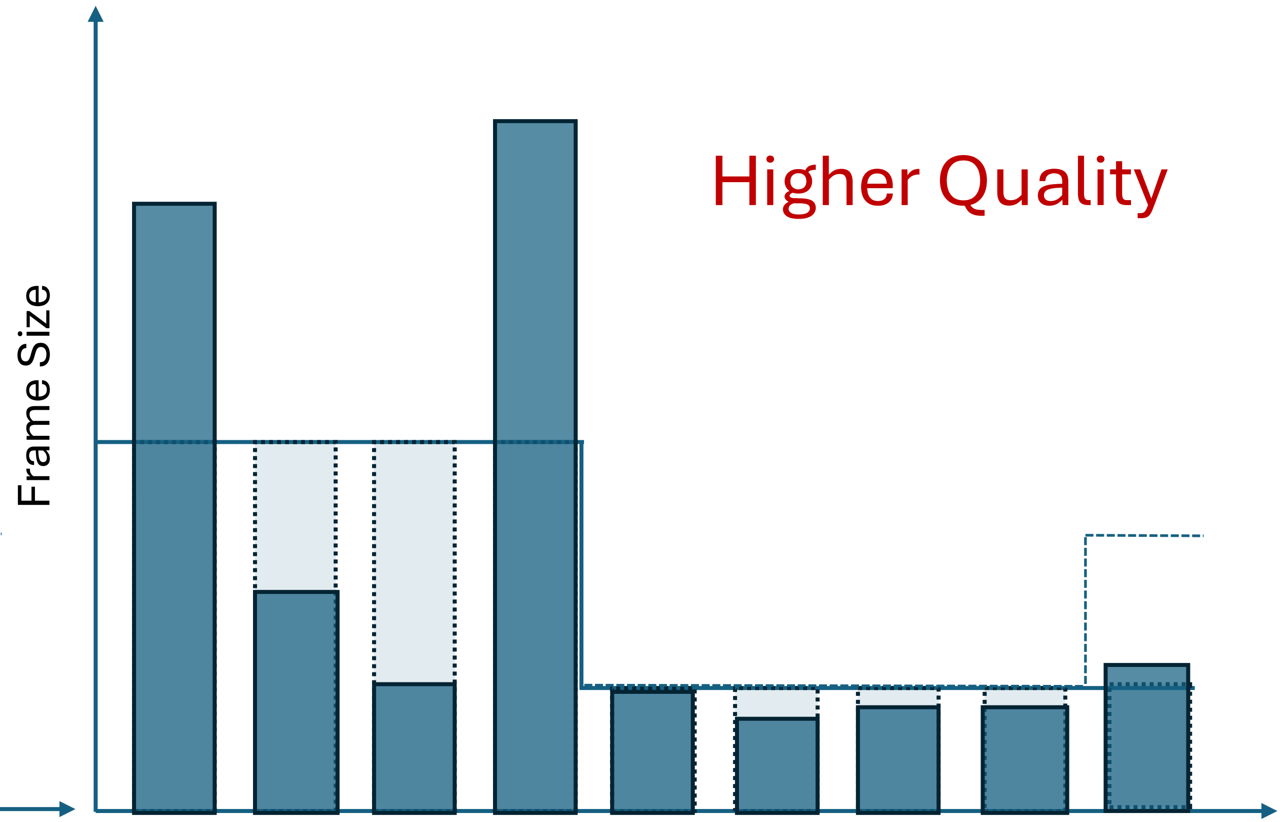
Why can't we directly reduce the original threshold?

MAE 2: Pre-reacting based on CCA internal state probing

Why can't we directly reduce the original threshold?

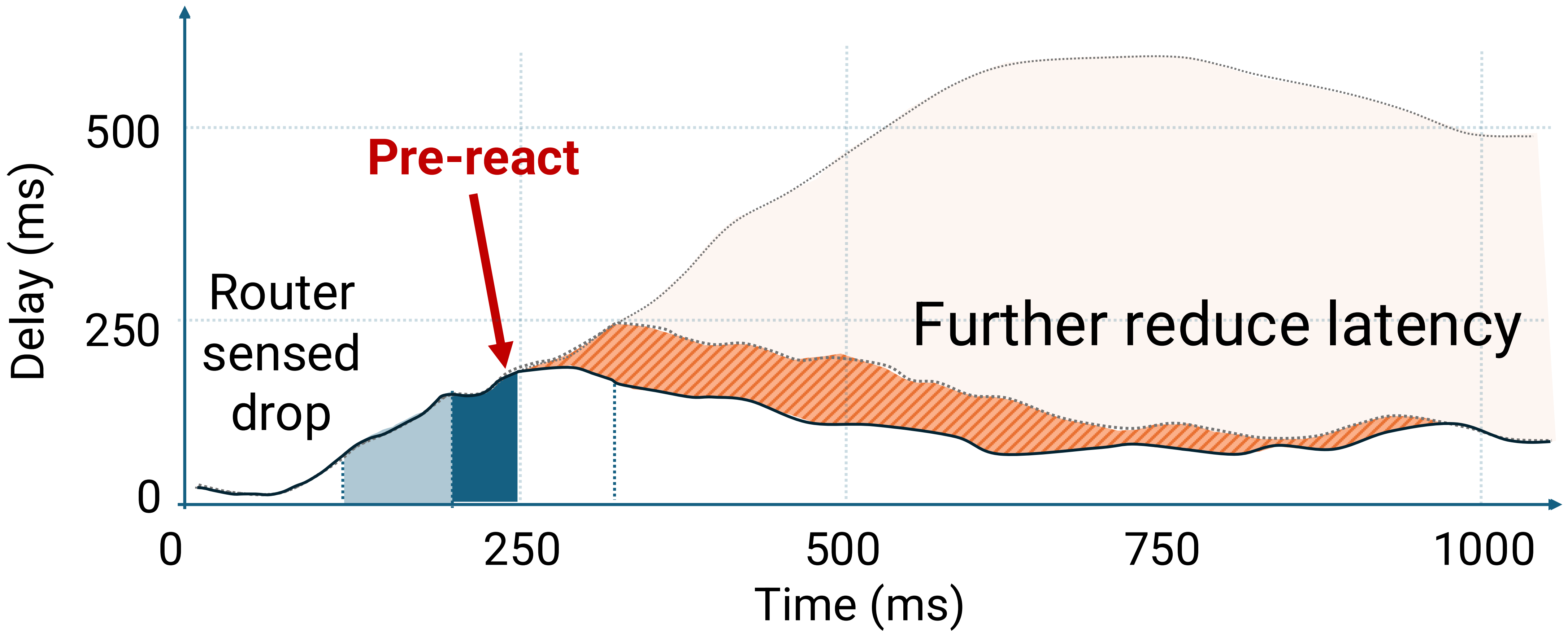


Directly reduce original threshold



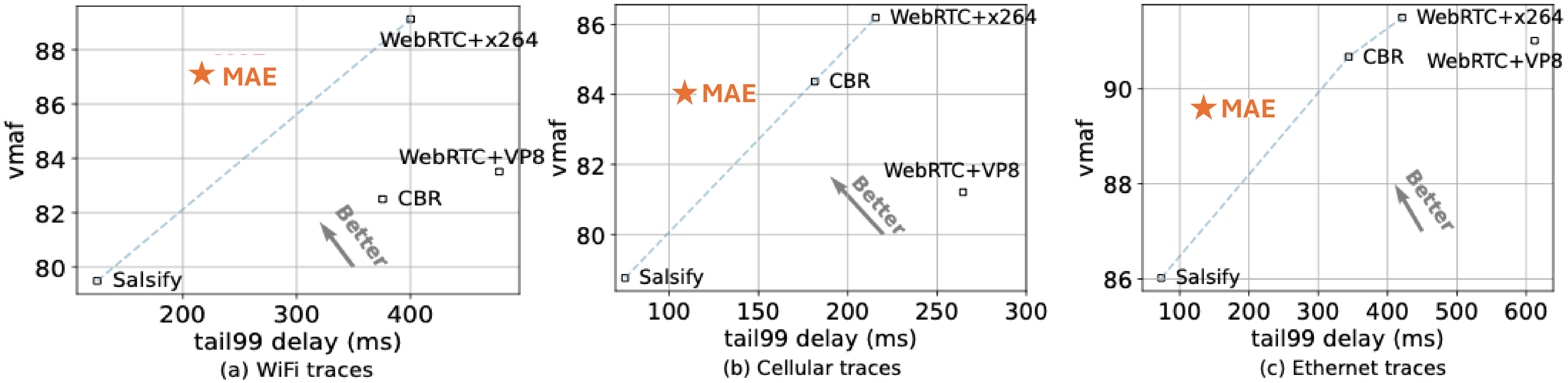
Only encoder adapts virtual buffer

MAE 2: Pre-reacting based on CCA internal state probing



Detailed reaction time when bandwidth drops from 10Mbps to 1Mbps.

Empirical performance across diverse networks



Overall Stalls

86.2% stall reduction.

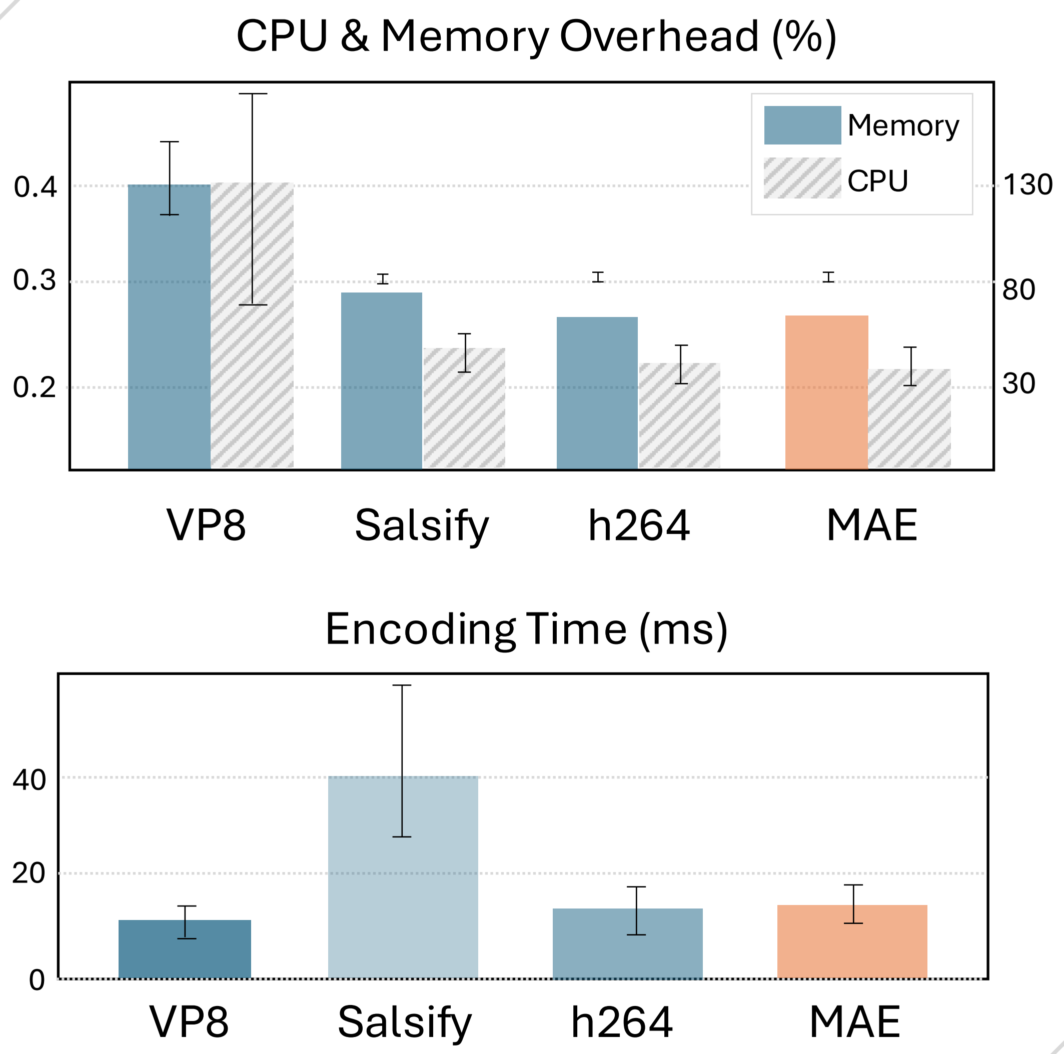
Global stall ratio drops massively from 3.8% to 0.69%.

High-Motion Quality Retention

11%.

In high-motion gaming content, MAE outperforms Salsify's VMAF by 11% by intelligently allocating bits during stable bandwidth periods.

Production-Ready: Zero System Overhead



Architectural Generality

Bandwidth Utilization

At **most 4%** reduction, barely impacts video quality.

CCA Agnostic Integration

Extended to Copa CCA, reducing **47.5%** stall ratio.

Takeaways

1. The **slow-response encoder** is the *core bottleneck* behind existing playback stalls.
2. The encoder possesses many **controllable potential**; treating it as a black box fails to leverage its inherent capability.
3. Current congestion control algorithms have the potential to **pre-react** before actual bandwidth drop signals emerge.
4. Our MAE framework requires **no modifications** to existing encoders, congestion control algorithms, or network protocols. It reduces stall rate by **86.2%** while preserving identical quality.

Thank you!

Project Website

https://github.com/hkust-spark/sparkrtc

Personal Website

https://huameng15.github.io